

Rustaceans

✦ Member-only story

Rust



Concurrency



Programming



Systems Programming



Memory Management



Epoch-Based Reclamation explained

With code in Rust



Enzo Lombardi



Following



10 min read · Aug 26, 2026



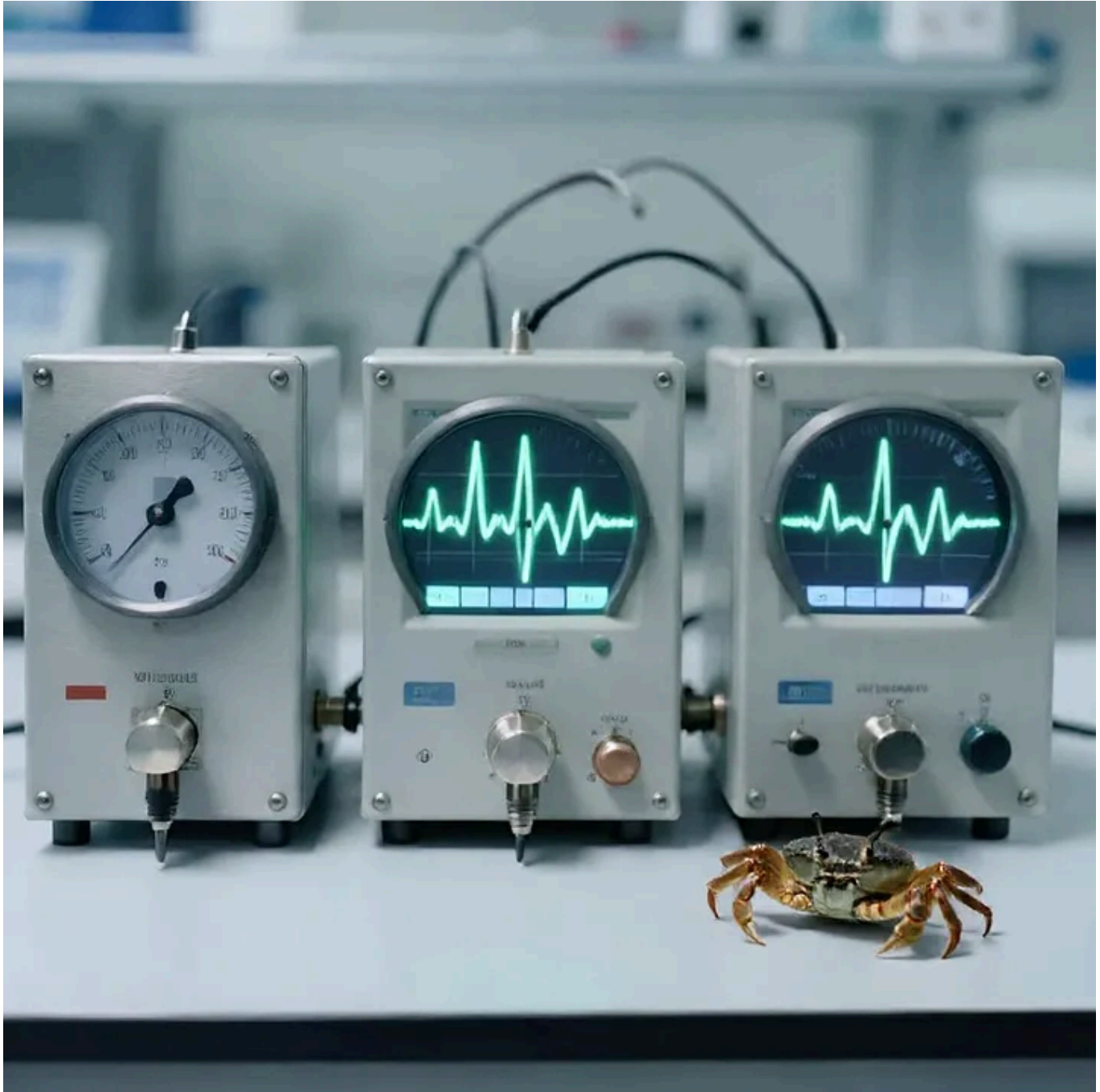
10



Writing a lock-free stack is a pleasant afternoon. You reach for a compare-and-swap, you loop until it succeeds, and you have a data structure that several threads can hammer without ever taking a lock. Then you try to free a node, and the afternoon ends.

The problem is not the algorithm. The problem is that lock-free code has no moment where a thread can say “nobody is looking at this any more”. A thread pops a node, unlinks it from the list, and holds the only pointer to it. That sounds like ownership. It is not. Another thread may have loaded the same pointer a nanosecond earlier and be about to dereference it.

`Box::from_raw` at that instant is a use-after-free, and the window is exactly as wide as the gap between two instructions on two different cores.

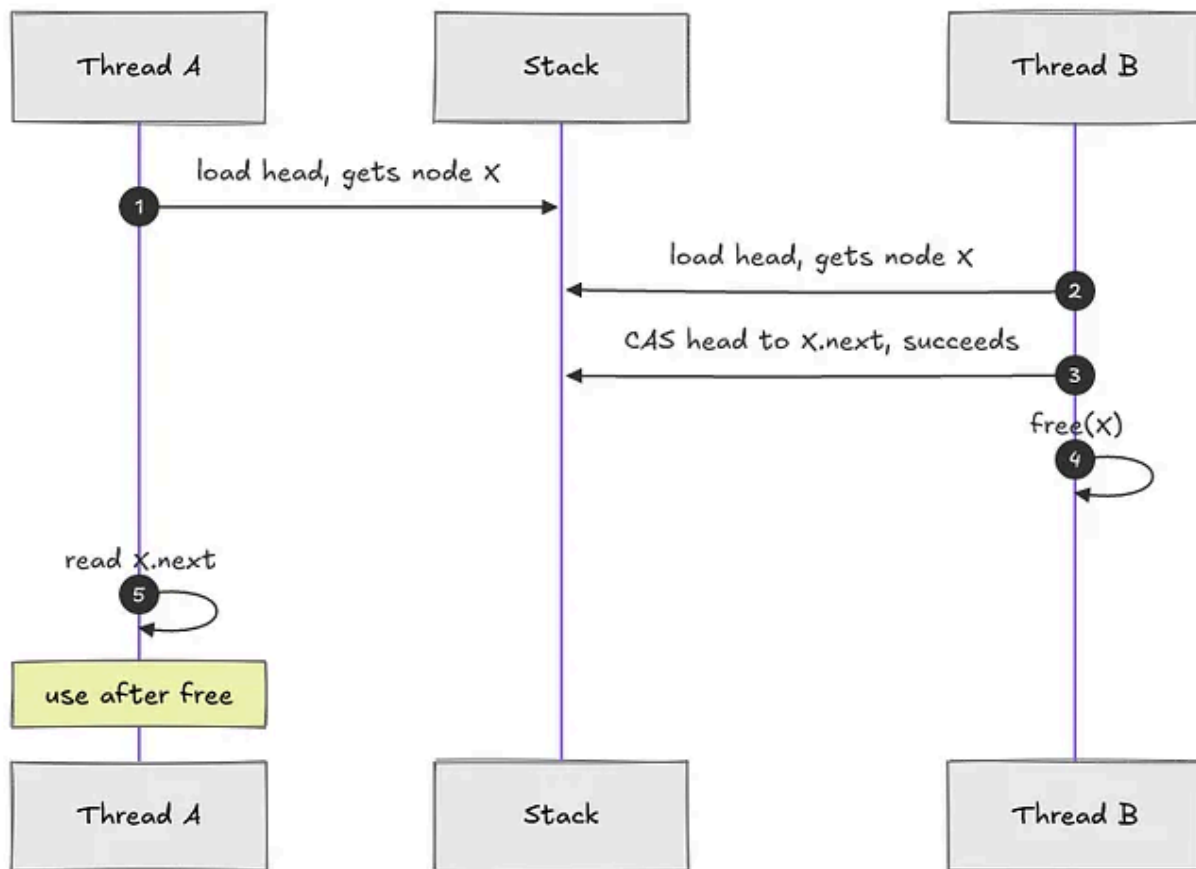


Epoch-based reclamation is the answer that most production Rust concurrency crates settled on. It does not try to work out whether a specific node is still referenced. It works out something coarser and far cheaper: whether every thread that could possibly have been holding old pointers has

since started fresh. Once that is true, everything retired before they started fresh is safe to free, all at once.

The race you have to lose

Consider a Treiber stack, the simplest lock-free structure there is. `pop` loads the head pointer, reads the node's `next` field, and swings the head to `next` with a compare-and-swap. Two threads running that concurrently produce the following interleaving with no effort at all.



<https://enzolombardi.net>

Thread B won the compare-and-swap, so it believes it owns node `X`. It is right about ownership and wrong about safety. Thread A is still holding a pointer it loaded legitimately, and the language cannot tell it to stop.

A garbage collector solves this by knowing about every reference. Reference counting solves it by paying an atomic increment on every read, which is exactly the cost lock-free code was trying to avoid, and which has its own chicken-and-egg problem: you have to dereference the pointer to increment its count, and the pointer may already be dead. Hazard pointers solve it precisely by publishing each pointer a thread is about to read at the price of

Open in app ↗



Medium



Search



Write



Epoch-based reclamation takes the opposite trade. Reads are nearly free. The cost moves to the freeing side, and the price you pay is that memory is released in batches, a little later than strictly necessary.

Pinning, epochs, and the two-behind rule

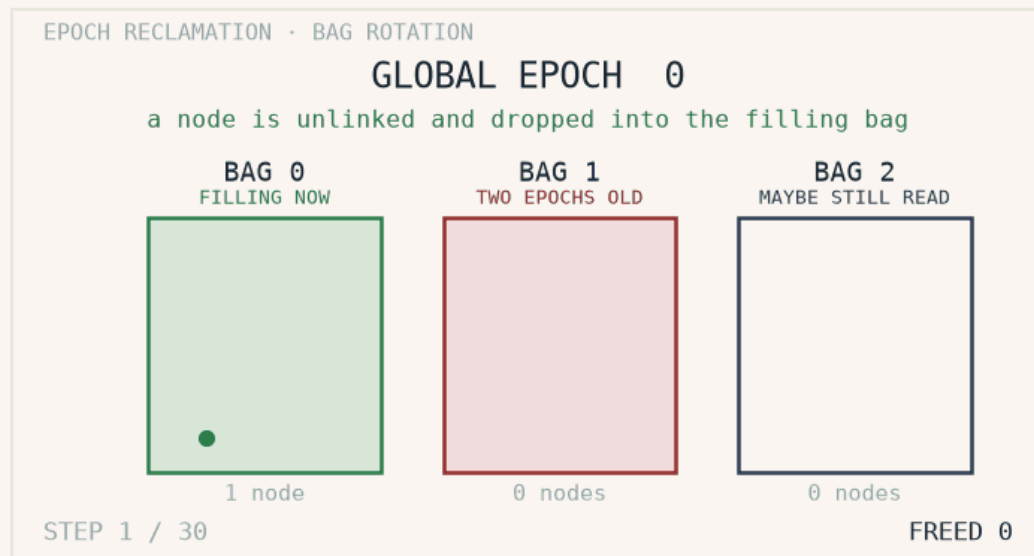
The scheme rests on a single global counter, the epoch, and one slot per thread. Before a thread touches any shared pointer it *pins* itself: it reads the global epoch and writes that value into its own slot. When it is done it writes a sentinel meaning “not participating”. That is the entire read-side protocol. One relaxed load, one store, and no fences on the fast path beyond what the store already implies.

When a thread unlinks a node it does not free it. It *retires* it, pushing the deallocation into a bag belonging to the epoch it is currently pinned in. Garbage therefore sorts itself by age without anyone timestamping anything.

Advancing is where the reasoning happens. Any thread may occasionally look at every participant's slot. If every pinned thread is sitting in the current epoch e , it bumps the global counter to $e + 1$. That bump is the interesting event, because of what it implies about the past: a thread pinned in e can only have loaded pointers that were reachable during e . It cannot have

loaded a pointer retired in $e - 1$, because to be retired the node must first have been unlinked, and unlinking happened before that thread pinned.

So when the epoch advances twice, from e to $e + 2$, no thread anywhere can still hold a pointer that was retired during epoch e . The bag for e is now provably dead, and can be emptied without checking anything.



The loop above is the engine's own state, sampled at every retirement and every advance during a real three-thread run, with the collection interval turned down so that several rotations fit into thirty steps. Watch what the bags do rather than what the counter does. A retirement always lands in the bag matching the current epoch. An advance changes nothing except which bag is which, and the bag that becomes free is always the one nobody has been allowed to touch for two full epochs. The `FREED` counter in the corner only ever moves in jumps, never one node at a time.

Three bags are enough, and that is not a coincidence. At any moment you need the one you are filling, the one that might still be observed, and the one you are about to empty. The fourth would be the first one again, which is why implementations index their bags with `epoch % 3`.

Building the engine

The global state is a counter, a registry of participants, and the three bags. A participant is nothing but an atomic word.

```
const INACTIVE: usize = usize::MAX;

struct Participant {
    epoch: AtomicUsize,
}

struct Global {
    epoch: AtomicUsize,
    participants: Mutex<Vec<Arc<Participant>>>,
    garbage: [Mutex<Vec<Deferred>>; 3],
}

type Deferred = Box<dyn FnOnce() + Send>;
```

A mutex around the participant registry looks like a betrayal of the whole premise, and it would be if reads took it. They do not. It is touched when a thread first registers and when someone attempts to advance the epoch, which happens on roughly one pin in sixty-four. The read path never sees it. Real implementations use a lock-free list here, but the difference is a constant factor on a rare operation, not a change in the algorithm.

Pinning is the whole read-side cost, and it is worth reading closely because there is so little of it.

```
pub fn pin() -> Guard {
    let participant = local();
    let e = global().epoch.load(Ordering::Relaxed);
    participant.epoch.store(e, Ordering::SeqCst);

    let n = PIN_COUNT.with(|c| { let n = c.get() + 1; c.set(n); n });
    if n % 64 == 0 {
        try_advance(e);
    }
    Guard { participant }
}

impl Drop for Guard {
    fn drop(&mut self) {
        self.participant.epoch.store(INACTIVE, Ordering::SeqCst);
    }
}
```

The load of the global epoch can be relaxed because reading a stale value is harmless: you would simply pin yourself into an epoch that has already been superseded, which delays reclamation and breaks nothing. The store into your own slot cannot be relaxed. It has to be visible before you load any shared pointer, otherwise a thread scanning the slots could conclude you are absent while you are already reading. Sequential consistency here buys the fence that makes the announcement stick, and it is the one genuinely expensive instruction in the read path.

`Guard` is an RAII token. Holding it means “I am pinned”, and dropping it means “I am done”, which lets the borrow checker enforce a rule the C version of this algorithm can only put in a comment. A function that returns a reference derived from shared memory can tie its lifetime to the guard, and the compiler will refuse to let that reference outlive the critical section.

Retiring appends to the bag for the epoch you are pinned in. Note that it takes `&self` on the guard, so you cannot retire anything without proving you are pinned.

```
impl Guard {  
    pub fn defer(&self, f: impl FnOnce() + Send + 'static) {  
        let e = self.participant.epoch.load(Ordering::Relaxed);  
        global().garbage[e % 3].lock().unwrap().push(Box::new(f));  
    }  
}
```

And the collector, which is where the two-behind rule turns into code.

```
fn try_advance(current: usize) {  
    {  
        let participants = global().participants.lock().unwrap();  
        let all_caught_up = participants.iter().all(|p| {  
            let e = p.epoch.load(Ordering::SeqCst);  
            e == INACTIVE || e == current  
        });  
        if !all_caught_up {  
            return;  
        }  
        if global().epoch  
            .compare_exchange(current, current + 1, Ordering::SeqCst, Ordering::Relaxed)  
            .is_err()  
        {  
            return;  
        }  
    }  
    let stale = std::mem::take(&mut *global().garbage[(current + 1) % 3].lock());  
    for f in stale {  
        f();  
    }  
}
```

Two details carry the correctness. A participant sitting at `INACTIVE` is not an obstacle, because an unpinned thread holds no pointers by construction; that is the difference between “slow” and “blocking” for this algorithm. And the bag being emptied is indexed $(\text{current} + 1) \% 3$, which is the same slot as $\text{current} - 2$. The bag you are about to free is the one that has been sitting untouched for two full epochs.

A stack that uses it

With the engine in place, the stack becomes almost boring, which is the point.

```
pub fn pop(&self) -> Option<T> {
    let guard = pin();
    loop {
        let head = self.head.load(Ordering::Acquire);
        if head.is_null() {
            return None;
        }
        // Safe to dereference: we are pinned, so even if another thread
        // unlinked and retired this node, it cannot have been freed yet.
        let next = unsafe { (*head).next };
        if self
            .head
            .compare_exchange(head, next, Ordering::AcqRel, Ordering::Relaxed)
            .is_ok()
        {
            let value = unsafe { (*head).value.take() };
            let doomed = SendPtr(head);
            guard.defer(move || {
                let doomed = doomed;
                drop(unsafe { Box::from_raw(doomed.0) });
            });
            return value;
        }
    }
}
```

Compare this to the sequence diagram from earlier. Thread A is the one reading `(*head).next` after thread B has already won the compare-and-swap. Under this scheme B did not free anything; it pushed a closure into a bag. That bag cannot be emptied until the epoch has advanced twice, and it cannot advance even once while A is pinned in the current epoch. A's dereference is safe, and it is safe without A having performed a single atomic operation on the node.

The closure needs one piece of Rust plumbing that is easy to trip over. A raw pointer is not `Send`, so it cannot travel in a `Box<dyn FnOnce() + Send>`. Wrapping it in a struct with an `unsafe impl Send` is the standard move, but since Rust 2021 a closure captures individual fields rather than whole structs, so `doomed.0` inside the closure would capture the bare pointer and defeat the wrapper. Rebinding `let doomed = doomed;` forces the capture of the whole wrapper. It looks like a no-op line, and removing it stops the code compiling.

Eight threads pushing and popping fifty thousand items each run clean, and the counter tells you how much work the collector actually did:

```
popped 400000 during the run, 0 left on the stack
global epoch reached 4333
```

Four thousand epoch advances across four hundred thousand retirements, so each advance released around ninety nodes in a batch. That ratio is the whole economic argument for the scheme. Reclamation is amortised over many operations instead of being paid one node at a time.

Where it costs you

This is a teaching implementation and it takes several shortcuts that production crates do not. The bags are behind mutexes rather than being thread-local with periodic flushing, so a burst of retirements from many threads will contend. Anything still sitting in a bag when the process ends is never freed, since nothing advances the epoch at shutdown. And `try_advance` runs on a fixed pin count rather than adapting to how much garbage has piled up.

The deeper cost is structural, and no implementation escapes it. A thread that pins itself and then blocks, on I/O, on a page fault, or on the operating system's scheduler deciding it has had enough CPU, freezes the epoch for everyone. Garbage accumulates in all three bags and reclamation stops until that thread comes back. Nothing is corrupted and nothing deadlocks, but memory usage grows without bound while the stall lasts. This is why the rule in every epoch-based codebase is that critical sections must be short and must never block. `crossbeam-epoch` mitigates it with a repin mechanism that lets long-running loops periodically release and reacquire, and with a cap on how much garbage a thread may accumulate before it is obliged to help collect.

Hazard pointers do not have this failure mode: a stalled thread pins exactly the handful of pointers it published, and everything else remains collectable. They pay for it with a per-access store and fence, and with a bounded number of protected pointers per thread. Neither approach dominates. Epochs are the right default when reads vastly outnumber retirements and critical sections are short, which describes most lock-free containers.

Using the real thing

In production you would reach for `crossbeam-epoch`, which is the same algorithm with the sharp edges filed off: lock-free participant registry,

thread-local garbage buffers, adaptive collection, and a typed pointer API that keeps the `unsafe` contained.

```
use crossbeam_epoch::{self as epoch, Atomic, Owned, Shared};

pub fn pop(&self) -> Option<T> {
    let guard = epoch::pin();
    loop {
        let head: Shared<Node<T>> = self.head.load(Acquire, &guard);
        let node = unsafe { head.as_ref() }?;
        let next = node.next.load(Acquire, &guard);
        if self.head.compare_exchange(head, next, AcqRel, Acquire, &guard).is_ok()
            unsafe {
                let value = std::ptr::read(&*node.value);
                guard.defer_destroy(head);
                return Some(value);
            }
    }
}
```

`Shared<'g, T>` carries the guard's lifetime, so the type system will not let a pointer escape the critical section. `defer_destroy` is `defer` with the destructor filled in. The shape of the loop is identical to the hand-rolled version, because the algorithm is identical; what you gain is that the parts most likely to be wrong have already been written by people who thought about them longer than an afternoon.

Why reclamation is the real lock-free problem

The literature on lock-free data structures spends most of its pages on the algorithms, and the algorithms are usually the easy part. A Treiber stack is fifteen lines. A Michael-Scott queue is forty. What makes them hard to ship is that removing a node from a structure is not the same as being allowed to

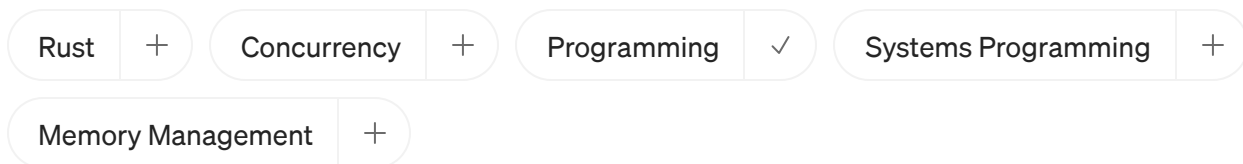
free it, and every scheme for closing that gap is a trade between read cost, reclamation latency, and how badly a stalled thread hurts you.

Epochs pick their corner of that space deliberately. Reads pay a store and a fence, freeing happens in batches, and a thread that sleeps while pinned costs you memory rather than correctness. Once you can name that trade, the crates stop looking like magic and start looking like a decision someone made, which is exactly the position from which you can tell whether it is the right decision for your code.

The full implementation, engine and stack and the eight-thread exercise, is [in this gist](#). It is a single file with no dependencies. Break it deliberately: change the bag count from three to two, or free immediately instead of deferring, and watch how long it takes for the failure to show up. The delay before it does is the best possible argument for not writing this by hand in production.

Want more like this?

I write regularly about Rust, design patterns, and performance tips. Follow me here [on Medium](#) to stay updated.





Written by Enzo Lombardi



1K followers · 167 following

Former Software Engineer at Google, still haven't found what I'm looking for. Retro computing, 6502 assembly, Rust. Full record: whois.enzolombardi.net

Following



Published in Rustaceans

3K followers · Last published 4 hours ago

Insights and Expertise from Rustaceans

Follow

No responses yet



Alex Mylnikov

What are your thoughts?

